

Documents structurés

Cours 8

Nassim ZELLAL

DOM - 2

Construction et transformation avec DOM

- Dans le présent cours, il s'agit de voir comment **construire** un « document structuré » de type XML, grâce à DOM en JAVA.
- Nous verrons également comment faire une **transformation** de document avec DOM en JAVA.

Construction avec DOM

- Avec le code fourni **CreateDoc.java**, nous pouvons construire le document XML « **create.xml** », qui est indenté avec la spécification d'un lien vers une DTD et de deux espaces de noms spécifiques avec leur URI.

Construction - create.xml

Ce document XML « create.xml » est construit par le code CreateDoc.java

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE Racine SYSTEM "exemple-dom.dtd">
<Racine xmlns:EN="http://example.net/english" xmlns:FR="http://example.net/french">
  <FR:langue>je suis le contenu du premier élément</FR:langue>
  <EN:langue>I am the content of the second element</EN:langue>
</Racine>
```

Construction du document « create.xml » avec DOM

```
import javax.xml.parsers.*;
```

```
import org.w3c.dom.DOMImplementation;
```

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();//Construction d'un parseur.  
DocumentBuilder parseur = factory.newDocumentBuilder();//Construction d'un parseur.  
//Syntaxe abrégée des deux précédentes lignes de code pour la création du parseur.  
//DocumentBuilder parseur = DocumentBuilderFactory.newInstance().newDocumentBuilder();  
//La méthode "getDOMImplementation()" retourne une instance de "DOMImplementation".  
//Cette étape est nécessaire, afin de créer par la suite un objet Document DOM.  
DOMImplementation domimp = parseur.getDOMImplementation();
```

Création d'un nouveau « Document »

```
//Nous pouvons placer un namespace sur l'élément racine (ici la valeur du premier argument est "null").  
//Nous pouvons aussi placer un objet DocumentType qui est la DTD associée au document XML (ici la valeur du  
//troisième argument est "null").  
//DocumentType dtd = domimp.createDocumentType("Racine",null,"exemple-dom.dtd");  
//Cette méthode permet de créer un objet Document DOM.  
Document doc = domimp.createDocument(null,"Racine",null);
```

Remarques :

- Nous pouvons placer l'URI d'un espace de noms (namespace) par défaut (voir cours 2) sur le nœud d'élément racine via le premier argument de la méthode createDocument(). Dans cet exemple, la valeur de ce premier argument est "null".
- Et nous pouvons aussi placer un objet DocumentType qui est la DTD associée au document XML. Dans cet exemple, la valeur de ce troisième argument est "null".
- createDocumentType("Racine",null,"exemple-dom.dtd");
- createDocument("URI" ,"Racine", dtd);

Création d'un nouveau « Document »

```
import org.w3c.dom.DocumentType;
```

```
DocumentType dtd = domimp.createDocumentType("Racine",null,"exemple-dom.dtd");  
Document doc = domimp.createDocument(null,"Racine",dtd);
```

Remarque :

- Dans la méthode createDocumentType(), nous pouvons placer l'identifiant public qu'on appelle aussi FPI (Format Public Identifier - voir cours 2) via le deuxième argument de la méthode createDocumentType(). Dans cet exemple, la valeur de ce deuxième argument est **"null"**.
- createDocumentType("html","-//W3C//DTD XHTML 1.0 Strict//EN","http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd");
- createDocument("http://www.w3.org/1999/xhtml" ,"html", dtd);

Création d'un nouveau « Document »

- Pour la création du lien vers la DTD, si on choisit l'implémentation sans la création de l'objet « dtd », on affichera ce lien comme suit :

- `tr.setOutputProperty(OutputKeys.DOCTYPE_SYSTEM, "exemple-dom.dtd");`

- Si on choisit l'implémentation avec la création de l'objet « dtd », on affichera ce lien via la méthode `getSystemId()` permettant de récupérer l'URI de la DTD.

- Si le **FPI** est défini, on peut le récupérer via la méthode `getPublicId()`.

- `tr.setOutputProperty(OutputKeys.DOCTYPE_SYSTEM, dtd.getSystemId());`

- `tr.setOutputProperty(OutputKeys.DOCTYPE_PUBLIC, dtd.getPublicId());`

Création d'un nouveau « Document »

```
//La méthode setXmlStandalone() permet de supprimer  
//l'attribut "standalone" (DTD)  
doc.setXmlStandalone(true);
```

DTD interne (XML autonome) / DTD externe (XML non autonome)

standalone="..."

valeur "**yes**" (interne) / "**no**" (externe)

Si cette ligne est commentée, cet attribut sera généré avec la valeur par défaut "**no**".

Si on souhaite générer cet attribut avec la valeur "**yes**", on ajoute cette ligne de code à la fin :

tr.setOutputProperty(OutputKeys.STANDALONE, "yes");

Création d'un nœud d'attribut - setAttribute()

```
//Récupération du nœud d'élément racine "Racine".  
Element rac = doc.getDocumentElement();  
String EN="EN:", FR= "FR:";  
//Création du premier nœud d'attribut (espace de nom spécifique - avec préfixe).  
rac.setAttribute("xmlns:FR", "http://example.net/french");  
//Création du deuxième nœud d'attribut (espace de nom spécifique - avec préfixe).  
rac.setAttribute("xmlns:EN", "http://example.net/english");
```

Chaque attribut contient le préfixe d'un « **espace de nom spécifique** » du nœud d'élément racine « Racine ».

Création d'un nœud élément - createElement() / ajout d'un nœud d'élément - appendChild()

```
//Création d'un premier nœud d'élément.  
Element prem_ele_FR = doc.createElement(FR+"langue");  
//Ajout de ce premier nœud d'élément au nœud d'élément racine.  
rac.appendChild(prem_ele_FR);  
//Création d'un deuxième nœud d'élément.  
Element prem_ele_EN = doc.createElement(EN+"langue");  
//Ajout de ce deuxième nœud d'élément au nœud d'élément racine.  
rac.appendChild(prem_ele_EN);
```

À chaque fois qu'on crée un élément avec la méthode « createElement() », on l'ajoute au nœud d'élément racine avec la méthode « appendChild() ».

Création d'un nœud texte - createTextNode()

```
//Création d'un premier nœud de texte et ajout de ce noeud au premier élément "langue".  
prem_ele_FR.appendChild((doc.createTextNode("je suis le contenu du premier élément")));  
//Création d'un deuxième nœud de texte et ajout de ce noeud au deuxième élément "langue".  
prem_ele_EN.appendChild((doc.createTextNode("I am the content of the second element")));
```

Création d'un nœud de texte (text node) pour chaque nœud d'élément « langue ».

Préparation de la sortie sur fichier

```
import javax.xml.transform.dom.DOMSource;
```

```
import javax.xml.transform.stream.StreamResult;
```

```
//Création de l'objet source, qui est une instance  
//de la classe "javax.xml.transform.dom.DOMSource".  
DOMSource ds = new DOMSource(doc);  
//Création de l'objet résultat, qui est instance  
//de la classe "javax.xml.transform.stream.StreamResult".  
StreamResult res = new StreamResult("create.xml");  
//Nous pouvons utiliser la ligne de code ci-dessous  
//pour afficher le résultat sur la sortie standard  
//StreamResult res = new StreamResult(System.out);
```

Construction avec DOM - sortie sur fichier

```
import javax.xml.transform.TransformerFactory;
```

```
import javax.xml.transform.Transformer;
```

```
//La sortie sur fichier est vue par JAXP comme un cas particulier de transformation.
```

```
//Obtention d'une instance de "TransformerFactory".
```

```
TransformerFactory transform = TransformerFactory.newInstance();
```

```
//Création du transformateur "tr".
```

```
Transformer tr = transform.newTransformer();
```

```
//Syntaxe abrégée des deux précédentes lignes de code.
```

```
//Transformer tr = TransformerFactory.newInstance().newTransformer();
```

Pour produire notre document « create.xml », nous avons besoin de trois types d'objets :

1- un objet source (ds);

2- un objet résultat (res);

3- un objet transformateur (tr).

Construction avec DOM - sortie sur fichier

```
import javax.xml.transform.OutputKeys;
```

```
//Spécification de l'encodage.  
tr.setOutputProperty(OutputKeys.ENCODING, "ISO-8859-1");  
//Indentation automatique du code source XML.  
tr.setOutputProperty(OutputKeys.INDENT, "yes");  
//Paramétrer l'indentation  
tr.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");  
//Ligne de création du lien vers la DTD.  
tr.setOutputProperty(OutputKeys.DOCTYPE_SYSTEM, "exemple-dom.dtd");  
//Transformation du document source (source XML) en un résultat (fichier XML).  
tr.transform(ds, res);
```

Construction - create.xml

Le document « create.xml » est construit par le code CreateDoc.java et il exploite la méthode 1 pour la génération du lien vers la DTD.

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE Racine SYSTEM "exemple-dom.dtd">
<Racine xmlns:EN="http://example.net/english" xmlns:FR="http://example.net/french">
  <FR:langue>je suis le contenu du premier élément</FR:langue>
  <EN:langue>I am the content of the second element</EN:langue>
</Racine>
```

Transformation avec DOM

- Partir d'un document source « source.xml » pour produire un document cible « cible.xml ».
 - Pour faire cette transformation, nous allons utiliser le code **TransformDoc.java**
-

Transformation - source.xml

```
<?xml version="1.0" encoding="iso-8859-1" ?>
<Voiture marque="Renault" modèle="Safrane">
  <Carosserie couleur="rouge">
    <Capot>Un peu cabossé</Capot>
  </Carosserie>
  <Moteur>
    <Cylindres />
    <Allumage>Défectueux</Allumage>
  </Moteur>
  <Transmission type="automatique" nb_vitesses="5">
    <Boîte />
    <TrainAV />
    <TrainAR />
  </Transmission>
</Voiture>
```

Transformation - cible.xml (résultat)

```
<?xml version="1.0" encoding="UTF-8"?><Racine><texte>Défectueux</texte></Racine>
```

Exercice

- Écrire un programme en JAVA/DOM, qui effectue une transformation, en générant le document cible à partir du document source.
 - La source et la cible se trouvent dans le document : « exercice-cours-8.pdf ».
-